LaaS Node

Issue Fix Guide

NFS Exporting Parent ZFS Dataset Instead of User Subdatasets

March 17, 2026 | Version 1.0

Issue Overview

All user data written inside containers — installed applications, conda environments, files — was being stored in the parent ZFS dataset (datapool/users) instead of each user's individual subdataset (datapool/users/testuser2, datapool/users/testuser3). This meant per-user quotas were not being enforced, user data was not isolated, and ZFS showed incorrect usage per user.

Symptom	Expected	Actual
df -h /home/ubuntu inside container	Shows user's quota (e.g. 15GB or 25GB)	Shows full pool size (34GB)
sudo zfs list testuser3 USED	Grows as user installs software	Always shows 24K regardless of installs
datapool/users REFER	Near zero (just metadata)	Shows all user data (11.8GB+)
Per-user quota enforcement	Each user limited to their quota	No enforcement — shared pool used
User data isolation	Completely separate per user	All users writing to same parent dataset

Root Cause

> **ROOT CAUSE:** NFS was exporting the parent dataset /datapool/users instead of each user's individual subdataset. When the container mounted /mnt/nfs/users/testuser3 and wrote files, those writes went into the parent dataset's filesystem layer — bypassing the testuser3

> ZFS subdataset entirely. ZFS subdatasets only receive data when they are the direct NFS export target.

How ZFS + NFS Interaction Works

ZFS datasets behave like separate filesystems stacked on top of each other. When you create datapool/users/testuser3, ZFS mounts it at /datapool/users/testuser3 — effectively hiding the parent's view of that directory. However, NFS does not automatically follow this stacking. If NFS exports the parent (/datapool/users), clients see a flat directory view and writes go into the parent's storage layer, not the child datasets.

This is what happened:

```
# What NFS was exporting (WRONG):
/datapool/users  →  exports the entire parent

# What the container mount looked like:
/mnt/nfs/users/testuser3  →  appears correct

# But where data actually went:
datapool/users (parent dataset)  ←  all writes landed here
datapool/users/testuser3         ←  always empty (24K)
```

Diagnosis Commands

Run these to confirm the issue:

```
# Check ZFS dataset usage — if parent shows large REFER
# and subdatasets show 24K, data is in the wrong place
sudo zfs list
# BAD output example:
# datapool/users            11.8G  21.6G     11.8G  /datapool/users   ← all data here
# datapool/users/testuser3    24K  21.6G       24K  /datapool/users/testuser3  ← empty

# Check what NFS is actually exporting
sudo exportfs -v
# BAD output:
# /datapool/users  127.0.0.1(...)  ← exporting parent, not subdatasets

# Check /etc/exports
cat /etc/exports
# BAD:
# /datapool/users 127.0.0.1(rw,sync,no_subtree_check,no_root_squash)

# Verify testuser3 quota — if it shows 'none' or full pool size
# quota is not being applied
sudo zfs get quota datapool/users/testuser3
# BAD: quota = none  (means no quota set or not applied)

# Check df inside container Konsole
df -h /home/ubuntu
# BAD: shows 34G total (full pool) instead of user quota
```

The Fix

Step 1 — Fix /etc/exports to Export Subdatasets Individually

Replace the parent export with individual exports for each user subdataset:

```
# Remove old parent export and replace with per-user exports
sudo bash -c "cat > /etc/exports << 'EOF'
/datapool/users/testuser2 127.0.0.1(rw,sync,no_subtree_check,no_root_squash)
/datapool/users/testuser3 127.0.0.1(rw,sync,no_subtree_check,no_root_squash)
EOF"

# Verify
cat /etc/exports
# Must show individual subdataset paths, NOT /datapool/users
```

Step 2 — Create Per-User NFS Mount Points

```
sudo mkdir -p /mnt/nfs/users/testuser2
sudo mkdir -p /mnt/nfs/users/testuser3
```

Step 3 — Unmount Old NFS, Apply New Exports, Remount

```
# Unmount old parent NFS mount
sudo umount -f /mnt/nfs/users 2>/dev/null || true

# Apply new exports
sudo exportfs -ra
sudo systemctl restart nfs-kernel-server

# Mount each subdataset directly
sudo mount -t nfs4 127.0.0.1:/datapool/users/testuser2 /mnt/nfs/users/testuser2
sudo mount -t nfs4 127.0.0.1:/datapool/users/testuser3 /mnt/nfs/users/testuser3

# Verify exports are correct
sudo exportfs -v
# GOOD output:
# /datapool/users/testuser2  127.0.0.1(...)
# /datapool/users/testuser3  127.0.0.1(...)
```

Step 4 — Update fstab for Persistence Across Reboots

```
# Remove old entry
sudo sed -i '/datapool\/users /d' /etc/fstab

# Add per-user entries
echo '127.0.0.1:/datapool/users/testuser2 /mnt/nfs/users/testuser2 nfs4 defaults 0 0' |
sudo tee -a /etc/fstab
echo '127.0.0.1:/datapool/users/testuser3 /mnt/nfs/users/testuser3 nfs4 defaults 0 0' |
sudo tee -a /etc/fstab

# Verify
cat /etc/fstab | grep nfs
```

Step 5 — Clean Old Data from Parent Dataset

Data that was incorrectly written to the parent dataset must be removed. This requires temporarily unmounting the subdatasets to expose what's underneath:

```
# Unmount subdatasets to expose parent data
sudo zfs unmount datapool/users/testuser2
sudo zfs unmount datapool/users/testuser3

# Delete old data from parent directories
sudo rm -rf /datapool/users/testuser2/*
sudo rm -rf /datapool/users/testuser2/.[!.]*  2>/dev/null || true
sudo rm -rf /datapool/users/testuser3/*
sudo rm -rf /datapool/users/testuser3/.[!.]*  2>/dev/null || true

# Remount subdatasets
sudo zfs mount datapool/users/testuser2
sudo zfs mount datapool/users/testuser3

# Verify parent is now clean
sudo zfs list
# GOOD output:
# datapool/users      ~100K  33G    ~40K  /datapool/users   ← near zero
# datapool/users/testuser2  24K  15G    24K                 ← empty, correct
# datapool/users/testuser3  24K  25G    24K                 ← empty, correct
```

> ⚠ **WARNING:** Cleaning old data from the parent dataset will delete any previously installed applications and user files. This is unavoidable if data landed in the wrong place. Going forward, all new installs will correctly land in the subdataset.

Step 6 — Set Correct Quotas

```
sudo zfs set quota=15G datapool/users/testuser2
sudo zfs set quota=25G datapool/users/testuser3

# Verify
sudo zfs get quota datapool/users/testuser2
sudo zfs get quota datapool/users/testuser3
```

Step 7 — Fix Ownership

```
sudo chown 1000:1000 /datapool/users/testuser2
sudo chown 1000:1000 /datapool/users/testuser3
```

Verification — Confirm Fix is Working

```
# 1. ZFS usage should show near-zero in parent
sudo zfs list

# 2. Quotas correctly set
sudo zfs get quota datapool/users/testuser2
sudo zfs get quota datapool/users/testuser3

# 3. NFS exporting correct paths
sudo exportfs -v

# 4. Install something inside container, then check ZFS usage grows
# Inside container Konsole:
# dd if=/dev/zero of=~/testfile bs=1M count=500
# Then on host:
sudo zfs list datapool/users/testuser3
# USED should now show ~500MB — confirms data going to correct dataset

# 5. df inside container shows correct quota
# Inside container Konsole:
df -h /home/ubuntu
# Must show: 25G total (or 15G for testuser2) — NOT 34G (full pool)
```

> ✅**CORRECT:** When df -h /home/ubuntu shows the user's quota (not the full pool size) and sudo zfs list shows USED growing in the subdataset — the fix is working correctly.

How to Avoid This in Future

Rule	Detail
Always export subdatasets in NFS, never the parent	/etc/exports must list each user's subdataset path individually
Always mount subdatasets directly in fstab	Each user gets their own mount point — never mount the parent and use subdirectories
Verify after setup with a write test	Write a test file inside container, check sudo zfs list shows USED growing in the subdataset
Set quotas before first container launch	If quota is not set, user can fill the entire pool

Rule	Detail
When adding a new user	Create ZFS dataset, set quota, export in NFS, add fstab entry, create mount point — all four steps required

Correct Architecture Reference

This is the correct relationship between ZFS, NFS, and containers:

```
ZFS Pool:
  datapool/users/testuser2  (quota=15G)  →  mounted at /datapool/users/testuser2
  datapool/users/testuser3  (quota=25G)  →  mounted at /datapool/users/testuser3

NFS Exports (/etc/exports):
  /datapool/users/testuser2  →  exported to 127.0.0.1
  /datapool/users/testuser3  →  exported to 127.0.0.1

Host NFS Mounts (/etc/fstab):
  127.0.0.1:/datapool/users/testuser2  →  /mnt/nfs/users/testuser2
  127.0.0.1:/datapool/users/testuser3  →  /mnt/nfs/users/testuser3

Docker Container Mounts:
  -v /mnt/nfs/users/testuser2:/home/ubuntu  (for testuser2 container)
  -v /mnt/nfs/users/testuser3:/home/ubuntu  (for testuser3 container)

Result:
  Container writes to /home/ubuntu
  → goes to /mnt/nfs/users/testuser3 (NFS)
  → goes to /datapool/users/testuser3 (ZFS subdataset)
  → quota enforced, isolated, correct
```